

Tutorial

React Login Interface

Ngày: 18/06/2026
Công nghệ: React 18 + Vite

Mục lục

1	Tổng quan dự án	2
1.1	Cấu trúc thư mục	2
1.2	Thiết lập TypeScript cho Vite (nếu dự án đang là JSX)	2
1.3	Luồng dữ liệu tổng quát	3
2	Flowchart: Luồng đăng nhập	3
2.1	Flowchart toàn bộ quá trình	4
2.2	Flowchart: Xử lý Promise	5
3	Bước 1 — Component <code>InputField.tsx</code> (Props + Type)	5
3.1	Code	5
3.2		6
3.3	Bảng props của <code>InputField</code>	7
4	Bước 2 — Component <code>LoginForm.tsx</code> (Props + <code>useState</code> + Type)	7
4.1	Code	7
4.2		9
4.3	Flowchart: Luồng validate trong <code>LoginForm</code>	10
5	Bước 3 — <code>App.tsx</code> (<code>useState</code> + <code>Promise</code> + <code>async/await</code> + Type)	10
5.1	Code	10
5.2	mô tả	12
5.3	<code>Promise</code> thuần vs <code>async/await</code>	13
6	Bước 4 — Luồng state trong <code>App</code>	13
6.1	Sơ đồ chuyển trạng thái (State Machine)	13
6.2	Bảng trạng thái	14
7	Tổng kết	14

1. Tổng quan dự án

1.1. Cấu trúc thư mục

```
src/
+-- App.tsx          <-- Component gốc (root)
+-- App.css          <-- Styles
+-- main.tsx         <-- Entry point
+-- components/
    +-- LoginForm.tsx <-- Form dạng nhập
    '-- InputField.tsx <-- Input tại sử dụng
```

1.2. Thiết lập TypeScript cho Vite (nếu dự án đang là JSX)

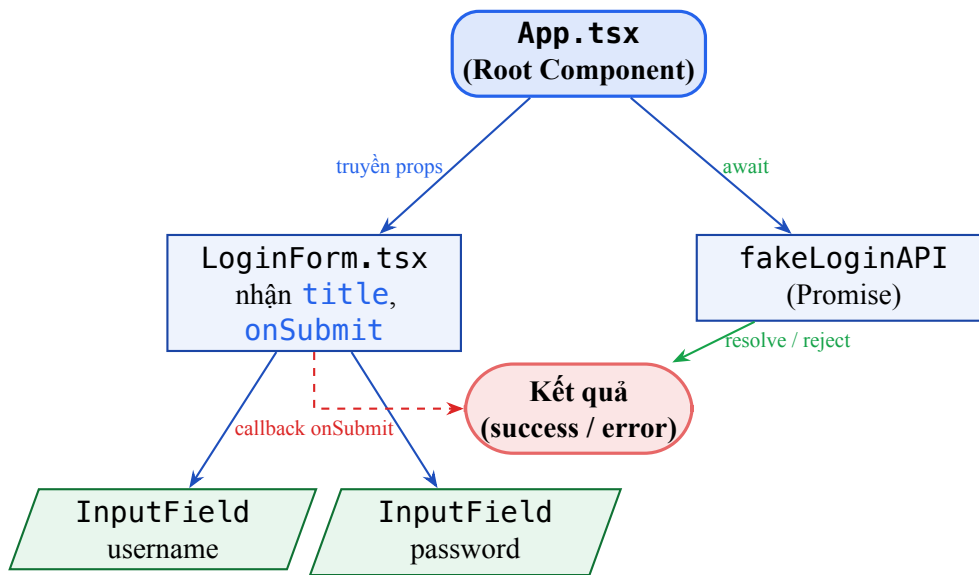
```
1 # 1) Cài TypeScript và type definitions
2 npm install -D typescript @types/react @types/react-dom
3
4 # 2) Đổi tên file
5 src/main.jsx -> src/main.tsx
6 src/App.jsx -> src/App.tsx
7 src/components/*.jsx -> *.tsx
8
9 # 3) Khởi tạo tsconfig nếu chưa có
10 npx tsc --init
11
12 # 4) Chạy lại dự án
13 npm run dev
```

Listing 1: Các bước chuyển React Vite sang TypeScript

Lưu ý

Trong code TypeScript bên dưới, trọng tâm là:
interface cho props, **type** cho union trạng thái, và **type** cho event/Promise để bắt lỗi sớm ngay lúc code.

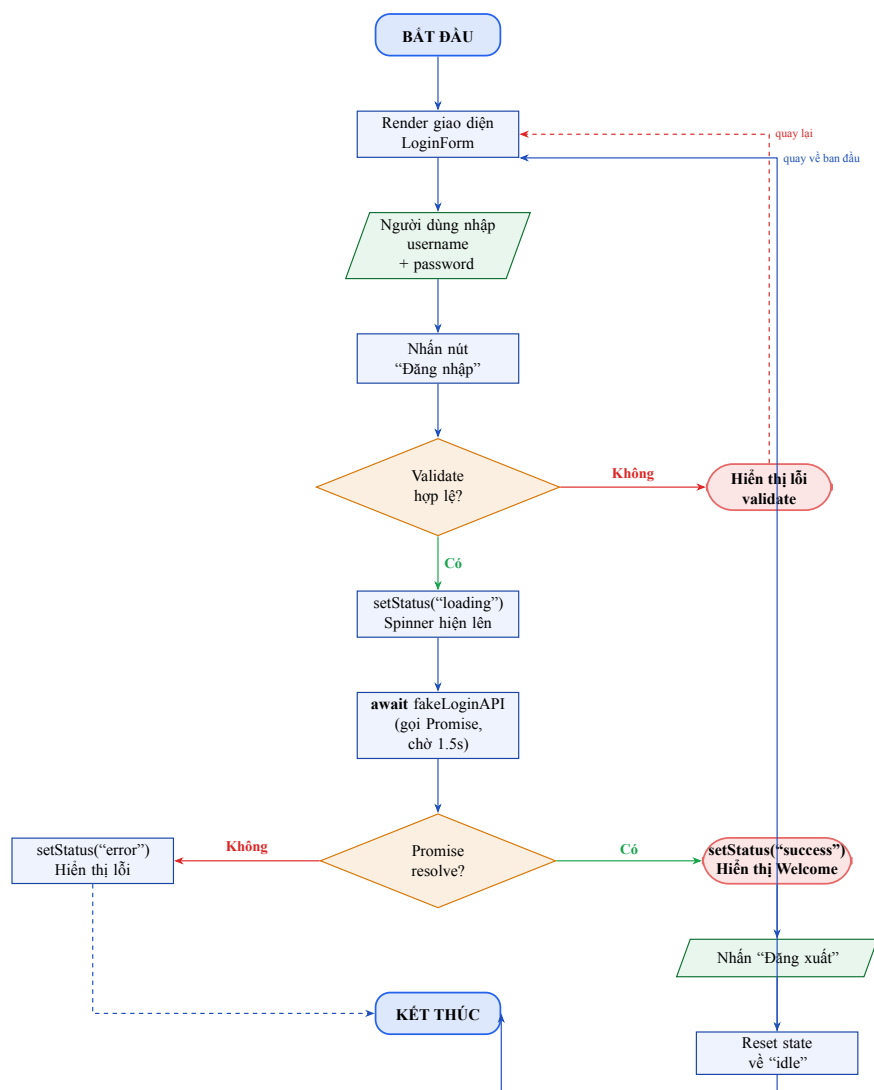
1.3. Luồng dữ liệu tổng quát



Hình 1: Luồng dữ liệu giữa các components

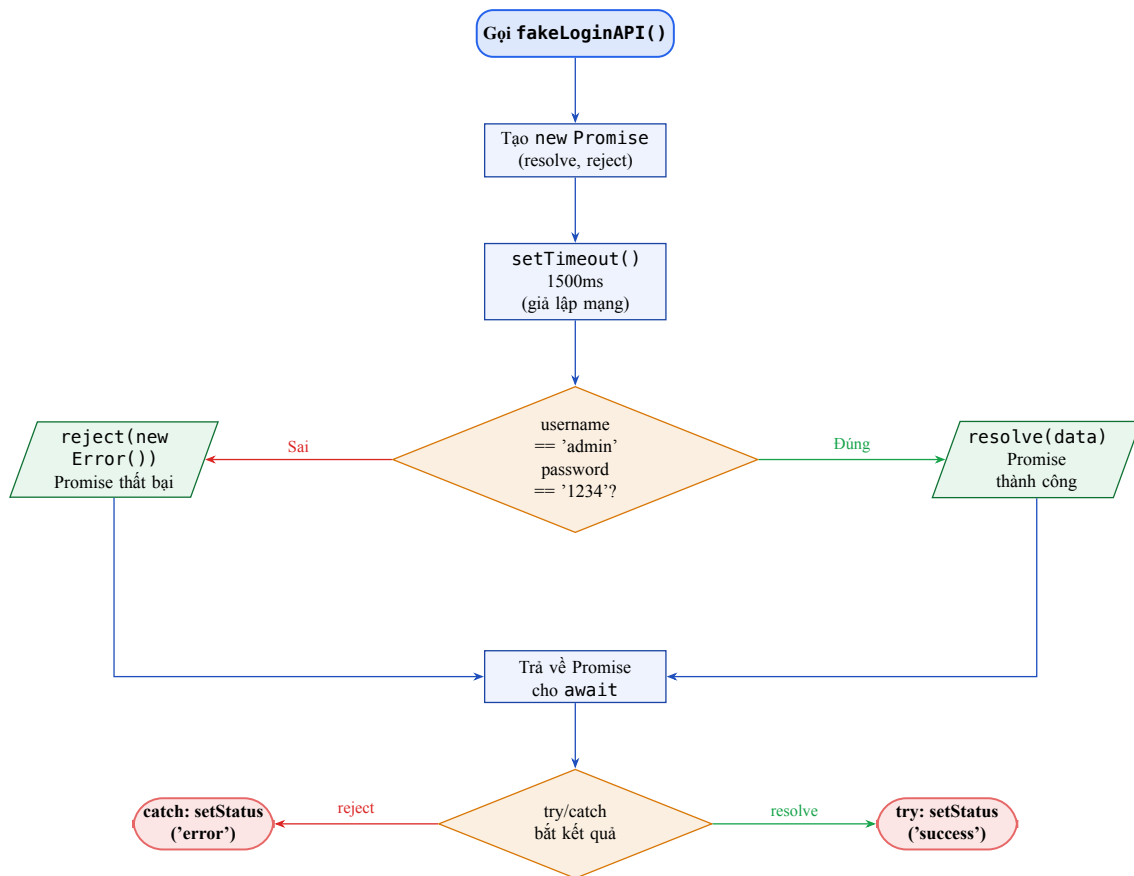
2. Flowchart: Luồng đăng nhập

2.1. Flowchart toàn bộ quá trình



Hình 2: Flowchart toàn bộ luồng đăng nhập

2.2. Flowchart: Xử lý Promise



Hình 3: Flowchart xử lý Promise trong fakeLoginAPI

3. Bước 1 — Component InputField.tsx (Props + Type)

Props (Properties) là cơ chế truyền dữ liệu **từ component cha xuống component con** theo một chiều (one-way data binding). Props chỉ được đọc, không được thay đổi bên trong component con.

3.1. Code

```
1 import type { ChangeEventHandler } from 'react'
2
3 type InputType = 'text' | 'password'
4
5 interface InputFieldProps {
6   label: string
7   type: InputType
8   value: string
9   onChange: ChangeEventHandler<HTMLInputElement>
```

```

10   placeholder?: string
11   error?: string
12 }
13
14 // Component InputField -- nhan du lieu qua PROPS co type
15 function InputField({
16   label,
17   type,
18   value,
19   onChange,
20   placeholder,
21   error,
22 }: InputFieldProps) {
23   return (
24     <div className="input-group">
25       <label className="input-label">{label}</label>
26       <input
27         type={type}
28         value={value}
29         onChange={onChange}
30         placeholder={placeholder}
31         className={`input-field ${error ? 'input-error' : ''}`}
32       />
33       { /* Hien thi loi neu co */ }
34       {error && <span className="error-msg">{error}</span>}
35     </div>
36   )
37 }
38
39 export default InputField

```

Listing 2: src/components/InputField.tsx

3.2.

Bước 1. Khai báo type cho props (dòng 5–12)

`interface InputFieldProps { ... }` giúp TypeScript kiểm tra đúng kiểu dữ liệu.

Bước 2. Destructuring + type annotation (dòng 15–22)

`{ label, type, value, onChange, placeholder, error }`

Và gắn `: InputFieldProps` để đảm bảo props truyền vào hợp lệ.

Bước 3. Render `label` (dòng 25)

`{label}` — hiển thị text nhãn, nhận từ props.

Bước 4. Bind `value` và `onChange` (dòng 27–29)

`value={value}` là *controlled input* — React kiểm soát giá trị.

`onChange={onChange}` — hàm cập nhật state, được truyền từ cha.

Bước 5. Conditional class (dòng 31)

`'input-field ${error ? 'input-error' : ''}'`

Nếu có lỗi, thêm class CSS `input-error` để viền đỏ.

Bước 6. Hiển thị lỗi có điều kiện (dòng 34)

`{error && <span>...}` — nếu `error` có giá trị thì render, nếu không thì bỏ qua.

3.3. Bảng props của `InputField`

Prop	Kiểu	Bắt buộc	Mô tả
<code>label</code>	<code>string</code>	Có	Nhãn hiển thị trên ô input
<code>type</code>	<code>InputType</code>	Có	Union type: <code>'text' 'password'</code>
<code>value</code>	<code>string</code>	Có	Giá trị hiện tại (controlled)
<code>onChange</code>	<code>React.ChangeEventHandler</code>	Có	Hàm xử lý nhập cho <code>HTMLInputElement</code>
<code>placeholder</code>	<code>string?</code>	Không	Gợi ý hiển thị khi rỗng
<code>error</code>	<code>string?</code>	Không	Thông báo lỗi, <code>undefined</code> nếu không có

4. Bước 2 — Component `LoginForm.tsx` (Props + `useState` + Type)

`useState` là React Hook cho phép component lưu và cập nhật dữ liệu nội bộ.
Cú pháp: `const [value, setValue] = useState(giaTriKhoiTao)`
Khi gọi `setValue(...)` React sẽ **re-render** component với giá trị mới.

4.1. Code

```
1 import { useState, type FormEvent } from 'react'
2 import InputField from './InputField'
3
4 export interface LoginPayload {
5   username: string
6   password: string
7 }
8
9 type SubmitHandler = (
10   payload: LoginPayload
11 ) => Promise<void> | void
12
13 interface LoginFormProps {
14   title: string
15   onSubmit: SubmitHandler
16 }
17
18 interface LoginErrors {
19   username?: string
```



```

20 password?: string
21 }
22
23 // Component LoginForm -- nhan onSubmit va title qua PROPS co type
24 function LoginForm({ title, onSubmit }: LoginFormProps) {
25
26     // ===== useState: quan ly state noi bo cua form =====
27     const [username, setUsername] = useState<string>('')
28     const [password, setPassword] = useState<string>('')
29     const [errors, setErrors] = useState<LoginErrors>({})
30
31     // Ham validate don gian
32     const validate = (): LoginErrors => {
33         const newErrors: LoginErrors = {}
34         if (!username.trim()) newErrors.username = 'Vui long nhap ten
35             ...'
36         if (!password.trim()) newErrors.password = 'Vui long nhap mat
37             khau'
38         else if (password.length < 4) newErrors.password = 'Toi thieu
39             4 ky tu'
40         return newErrors
41     }
42
43     const handleSubmit = (e: FormEvent<HTMLFormElement>) => {
44         e.preventDefault() // Ngan form reload
45         trang
46         const validationErrors = validate()
47         if (Object.keys(validationErrors).length > 0) {
48             setErrors(validationErrors) // Cap nhat state
49             errors
50             return
51         }
52         setErrors({})
53         onSubmit({ username, password }) // Goi callback tu
54         PROPS
55     }
56
57     return (
58         <form className="login-form" onSubmit={handleSubmit}>
59             <h2 className="form-title">{title}</h2> /* Props title
60             */
61
62             <InputField
63                 label="Ten dang nhap"
64                 type="text"
65                 value={username} // state -> props
66                 onChange={(e) => setUsername(e.target.value)}
67                 placeholder="Nhap ten dang nhap..."
68                 error={errors.username}
69             />

```

```

64     <InputField
65         label="Mat khau"
66         type="password"
67         value={password} // state -> props
68         onChange={(e) => setPassword(e.target.value)}
69         placeholder="Nhập mật khẩu..."
70         error={errors.password}
71     />
72
73     <button type="submit" className="login-btn">
74         Đang nhập
75     </button>
76 </form>
77 )
78 }
79
80 export default LoginForm

```

Listing 3: src/components/LoginForm.tsx

4.2.

Bước 1. Khai báo 3 state (dòng 8–10)

State	Khởi tạo	Lưu gì
username	"	Chuỗi người dùng nhập vào ô username
password	"	Chuỗi người dùng nhập vào ô password
errors	{}	Object kiểu <code>LoginErrors</code> chứa lỗi validate

Bước 2. Type cho Props và Payload (dòng 4–21)

`LoginFormProps` và `SubmitHandler` đảm bảo callback nhận đúng dữ liệu `LoginPayload`.

Bước 3. Hàm `validate` (dòng 13–18)

Tạo object `newErrors`, thêm lỗi nếu ô trống hoặc quá ngắn.
Trả về object rỗng `{}` nếu hợp lệ.

Bước 4. `e.preventDefault()` (dòng 22)

Ngăn trình duyệt reload trang khi submit form HTML.

Bước 5. Cập nhật `errors` state (dòng 24)

`setErrors(validationErrors)` → React re-render, hiển thị lỗi.

Bước 6. Gọi callback `onSubmit` từ props (dòng 28)

`onSubmit({ username, password })` — truyền dữ liệu lên component cha.

Bước 7. Truyền state xuống props của `InputField` (dòng 36–44)

`value={username}` và `onChange={(e) => setUsername(e.target.value)}`

Tạo ra *controlled component*: state là nguồn sự thật duy nhất.

4.3. Flowchart: Luồng validate trong LoginForm

5. Bước 3 — App.tsx (useState + Promise + async/await + Type)

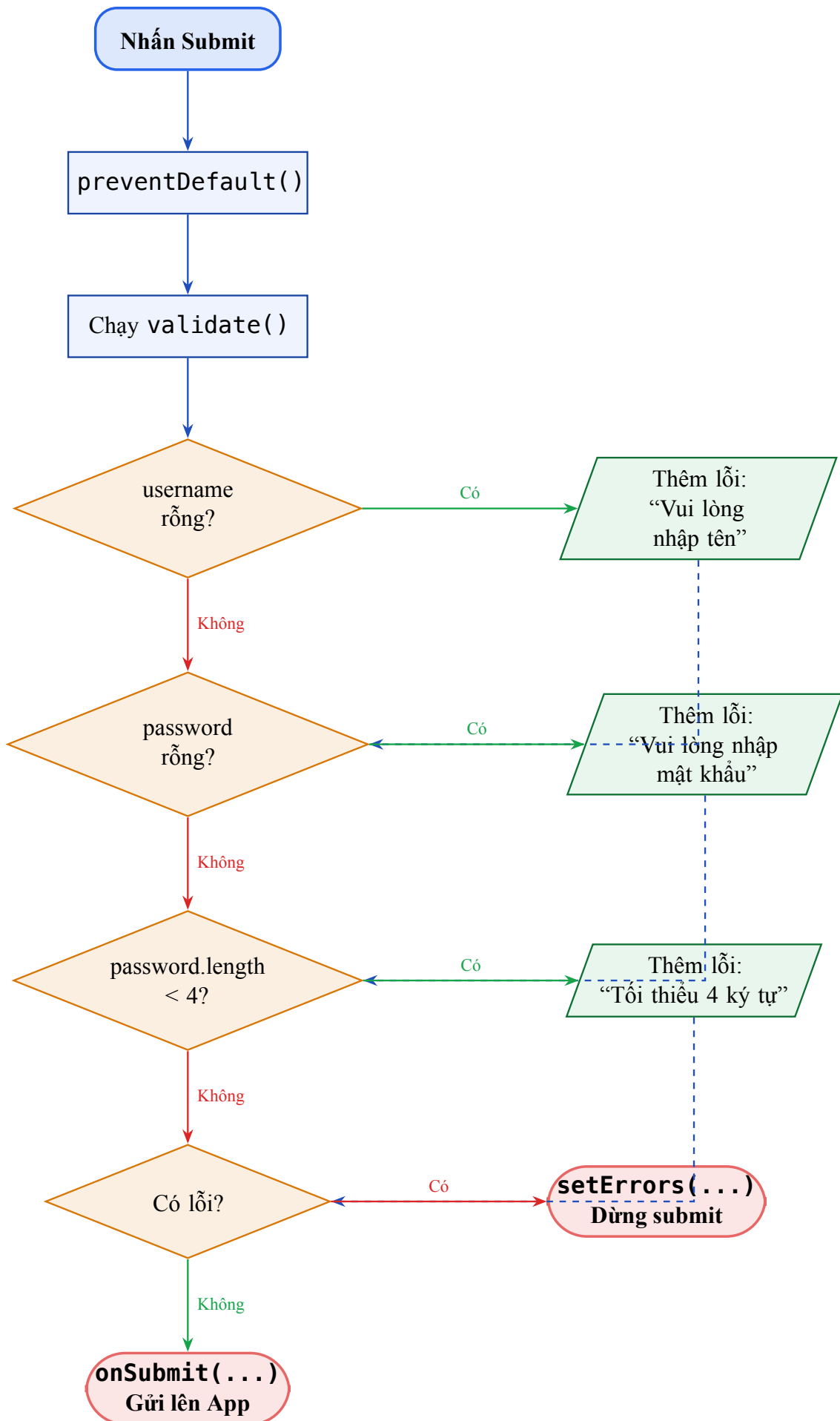
Promise đại diện cho một giá trị **chưa có ngay** (bất đồng bộ).
Promise có 3 trạng thái:

- **Pending** — đang chờ (chưa xong)
- **Fulfilled** (resolve) — thành công, có giá trị
- **Rejected** — thất bại, có lỗi

async/await là cú pháp giúp viết code bất đồng bộ **trông như đồng bộ**.
await chỉ dùng được bên trong hàm **async**.
Kết hợp với **try/catch** để bắt lỗi từ Promise bị reject.

5.1. Code

```
1 type LoginStatus = 'idle' | 'loading' | 'success' | 'error'
2
3 interface LoginPayload {
4   username: string
5   password: string
6 }
7
8 interface LoginResponse {
9   message: string
10 }
11
12 // ===== Ham gia lap API bang PROMISE =====
13 function fakeLoginAPI(username: string, password: string): Promise
14   <LoginResponse> {
15   return new Promise((resolve, reject) => { // (1) Tao Promise
16     // typed
17     setTimeout(() => { // (2) Gia lap do
18       // tre mang
19       if (username === 'admin' && password === '1234') {
20         resolve({ message: 'Xin chào, ${username}!' }) // (3)
21         Thanh cong
22       } else {
23         reject(new Error('Sai tài khoản hoặc mật khẩu!')) // (4)
24         That bai
25       }
26     })
27   }
```



```

21     }, 1500)
22   })
23 }
24
25 function App() {
26   // useState: quan ly trang thai cua app
27   const [status, setStatus] = useState<LoginStatus>('idle'
28   )
29   const [message, setMessage] = useState<string>('')
30   const [loggedUser, setLoggedUser] = useState<string | null>(null
31   )
32
33   // ===== async/await: xu ly bat dong bo =====
34   const handleLogin = async ({ username, password }: LoginPayload)
35     : Promise<void> => {
36     setStatus('loading')
37     setMessage('')
38
39     try {
40       // (6) await: cho Promise hoan thanh
41       const result = await fakeLoginAPI(username, password)
42       setStatus('success') // (7) Promise
43       resolve
44       setMessage(result.message)
45       setLoggedUser(username)
46     } catch (error) {
47       const errorMessage = error instanceof Error ? error.message
48       : 'Dang nhap that bai'
49       setStatus('error') // (8) Promise
50       reject
51       setMessage(errorMessage)
52     }
53   }
54   // ...
55 }

```

Listing 4: src/App.tsx – phân quan trong

5.2. mô tả

- (1) **Tạo Promise** — `new Promise((resolve, reject)=> \{...\})`
 Nhận 2 callback: `resolve` để báo thành công, `reject` để báo thất bại.
- (2) **setTimeout** — giả lập độ trễ mạng 1500ms.
 Trong thực tế, đây là `fetch('https://api.example.com/login')`.
- (3) **resolve** — gọi khi thành công, Promise chuyển sang *Fulfilled*.
 Giá trị truyền vào `resolve` sẽ là kết quả nhận được sau `await`.
- (4) **reject** — gọi khi thất bại, Promise chuyển sang *Rejected*.
 Lỗi sẽ bị bắt bởi `catch`.

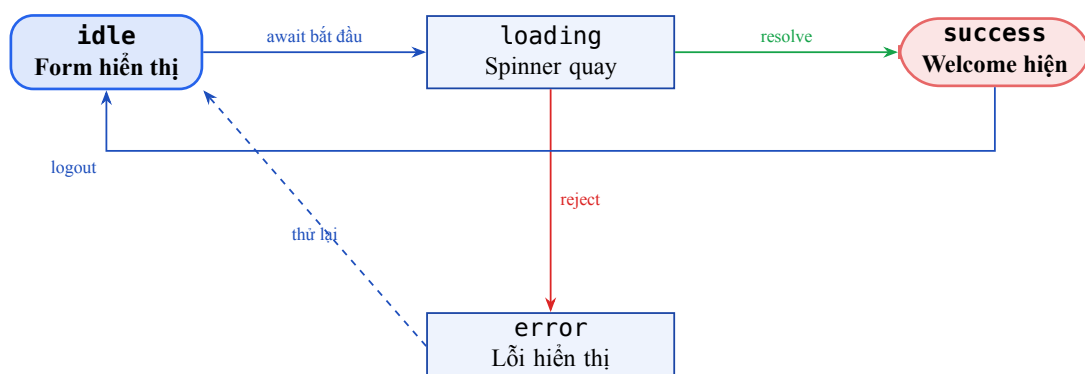
- (5) **async function** — khai báo `handleLogin` là hàm bất đồng bộ.
Hàm `async` luôn trả về một Promise.
- (6) **await** — dừng lại và chờ `fakeLoginAPI` trả về kết quả.
Không block UI — trình duyệt vẫn hoạt động bình thường trong lúc chờ.
- (7) **Type an toàn khi bắt lỗi** — `error instanceof Error`
Tránh lỗi TypeScript khi truy cập `error.message`.
- (8) **Xử lý thành công (try block)** — cập nhật state khi resolve.
- (9) **Xử lý thất bại (catch block)** — cập nhật state khi reject.

5.3. Promise thuần vs async/await

Promise	async/await
<pre>fakeLoginAPI(u, p) .then(result => { setStatus('success') }) .catch(err => { setStatus('error') })</pre>	<pre>try { const result = await fakeLoginAPI(u, p) setStatus('success') } catch (err) { setStatus('error') }</pre>
Đễ bị “callback hell” khi Đọc như code đồng bộ, dễ debug nhiều lớp	

6. Bước 4 — Luồng state trong App

6.1. Sơ đồ chuyển trạng thái (State Machine)

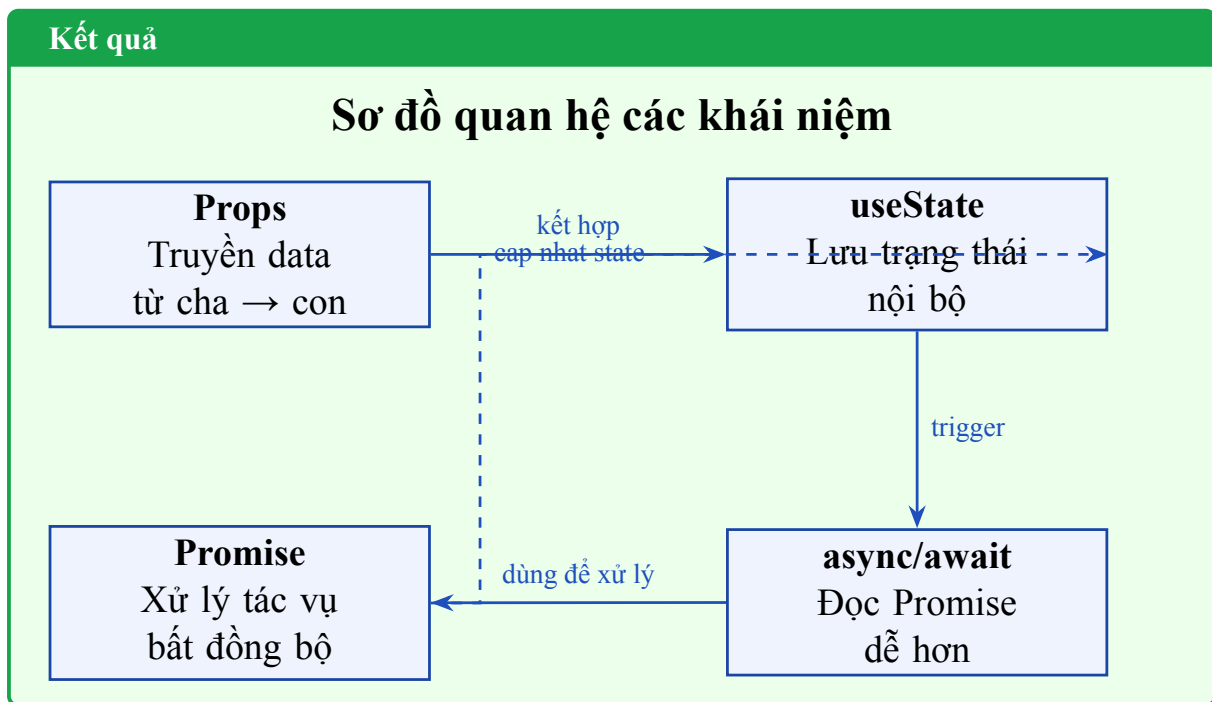


Hình 5: State machine của App.tsx

6.2. Bảng trạng thái

status	UI hiển thị	Trigger	Chuyển sang
idle	Form đăng nhập	Ban đầu / logout	loading
loading	Spinner + “Đang xử lý”	<code>await</code> bắt đầu	success / error
success	Welcome box	Promise resolve	idle (khi logout)
error	Thông báo lỗi đỏ	Promise reject	idle (thử lại)

7. Tổng kết



Khái niệm	Khi nào dùng
Props	Khi cần truyền dữ liệu / hàm xuống component con
useState	Khi component cần nhớ một giá trị và re-render khi đổi
Promise	Khi gọi API, đọc file, hay bất kỳ tác vụ mất thời gian
async/await	Luôn dùng thay cho <code>.then().catch()</code> cho dễ đọc